

Analysis: 3D Printing

The first thing we can notice is that if a printer has u units of ink left of a given color, we cannot use more than u units of that color. Moreover, this is the only restriction imposed by that value. So, we can summarize the input by saying we cannot use more than $C = \min(\mathbf{C}_1, \mathbf{C}_2, \mathbf{C}_3)$ units of cyan ink, $M = \min(\mathbf{M}_1, \mathbf{M}_2, \mathbf{M}_3)$ units of magenta ink, $Y = \min(\mathbf{Y}_1, \mathbf{Y}_2, \mathbf{Y}_3)$ units of yellow ink, or $K = \min(\mathbf{K}_1, \mathbf{K}_2, \mathbf{K}_3)$ units of black ink.

If $C + M + Y + K < 10^6$ then the case is impossible and we are done. Otherwise, we may need to use lower amounts of each color. We can simply go one color at a time, lowering the amounts of ink until we make the sum exactly 10^6 . Doing it one unit at a time works, but it is very slow. We can do better: in the same way as before, we can consider all the colors one at a time. Let S be the sum of the current amount of ink for the 3 colors not currently under consideration. If $S \geq 10^6$, we can simply set the amount of the current color to 0 and continue with the next one. If $S < 10^6$ we can lower the current color to $10^6 - S$ and finish immediately. This works because at all times we maintain the invariant that the total amount of ink we are considering is at least 10^6 units.

Analysis: Chain Reactions

Test Set 1

In Test Set 1 there are very few modules. This means that we can try all possible orders for the manual initiators, simulate the rules as explained in the statement to get the overall fun yielded by each order, and keep the maximum result from among those. Notice that modules that are pointed at the abyss and not pointed at by any other module contribute their own fun to the total no matter the order. Therefore, we can assume they all trigger in any specific order at the beginning. This brings down the number of orders to try from $N!$ to at most $(N/2)!$, which is a lot less.

Test Set 2

We can start by modeling the problem. We can see the input as a [rooted forest](#) where the parenting relationship models the pointed at relationship. Root nodes are modules that are pointed at the abyss.

As it is often the case for problems on trees, we can solve this one efficiently with a [divide and conquer](#) approach, and the aid of [memoization/dynamic programming](#) to keep the running time small.

Notice we can solve each tree (connected component) in the forest independently. Then, we can notice that the root of the tree is triggered by the first manual initiator. So, we can try all possibilities for the first manual initiator and eliminate the path between them and the root. That leaves a lot of separated subtrees that we can solve recursively.

Formally, let $F(i)$ be the fun of node i and $Ftree(t)$ be the fun value of a given subtree rooted at t . To compute $Ftree(t)$, if t is a single node we simply return its fun $F(t)$. Otherwise, we take the maximum (over all possible x) of $\text{fun}(x, t) + \sum_s Ftree(s)$ where $\text{fun}(x, t)$ is the maximum fun of all nodes in the path from x to t 's root and x is a leaf of this subtree and the summation is over all subtrees s whose root parent in the original tree is in the mentioned path.

The domain of F is equal to the number of subtrees, which is equal to the number of nodes of the tree. If we memoize F , the overall running time of computing it for all subtrees of a tree of size k is the domain size (k) times the time it takes to compute it for a single item in the domain, disregarding the cost of recursive calls. This is $O(k)$ because of the summation, which means an overall time complexity of $O(k^2)$. The worst case is when the entire input forest is a single tree, which means the time complexity of the algorithm overall is $O(N^2)$.

Test Set 3

To solve Test Set 3 we continue on our modelling from Test Set 2. We observe that the answer for $Ftree(t)$ can be broken down to $\max(\text{fun}(x, a), F(t)) + \sum_s Ftree(s)$ where a is one of t 's children and x is a leaf node. We also observe that to maximize this, we need to choose a leaf x which minimizes $\text{fun}(x, a)$. This way we guarantee that $\text{fun}(x, t)$ benefits from having $F(t)$ on the path, and that all other subtrees s have a maximum possible value. Otherwise if we pick a path other than the minimum, this means the minimum path will be considered as one of the other subtrees which reduces $\sum_s Ftree(s)$ reducing the final answer we can get.

Identifying x can be done using a DFS traversal solving each tree of size k in $O(k)$. Leading to an overall complexity of $O(N)$.

Analysis: d1000000

Test Set 1

There are multiple of ways to solve Test Set 1 of this problem. A particularly funny one is to throw the solution of an [old finals problem](#) at it (even the Test Set 1 solution of that problem works).

Test Set 2

Test Set 2 has very big numbers, so we need insights that are specific to this problem.

Insight 1. If a straight from A to B can be done, then one from 1 to $B - A + 1$ can be done as well using the same dice in the same order, since a die showing a number X can always be used to show number $X - A + 1$.

Insight 2. If a straight is done with a d_i showing number X and a d_j showing number $X + 1$ with $i > j$, we can build the same straight but using d_j for X and d_i for $X + 1$.

Insight 2b. Any straight that can be done, can also be done while using the dice in non-decreasing order of number of faces.

Combining insights 1 and 2b gives an algorithm: start by sorting the dice. Then, in that order, try to extend the current straight if possible. Or, in pseudo-code:

```
maximum_straight_length(S):  
    sort(S)  
    length = 0  
    for si in S:  
        if si > length: length += 1  
    return length
```

This algorithm requires only linear time beyond sorting the input, which means $O(N \log N)$ overall. This is fast enough to pass Test Set 2.

Because the dimensions of the punched cards we need to print are so small, one option would be to store all the ASCII pictures in an array and just print the requested cards. However, we can make use of nested [for-loops](#) to generate the art for any sized punched card.

As some of you may have already discovered, for this year's qualification round, you could submit solutions in *Punched Card Python*. The following is a solution to *Punched Cards* written using *Punched Card Python*:

```
.split())\n o,e='+-*c++\\n','|.'*c+'|\\n'\n print(f'
```

[illegible]

Analysis: Twisty Little Passages

In general, we can't just teleport to every room, because N can be much larger than K . We could teleport to a randomly-chosen subset of the rooms, calculate the average degree (number of adjoining passages) of those rooms we've visited, and assume that this is a good estimate of the average degree of all the rooms. The number of passages is half the sum of the room degrees (since each passage connects two rooms.)

When can this approach yield a poor estimate? If a small subset of the rooms have a degree much higher or much lower than the median degree, then we might not visit any of them, and our estimate would be too high or too low.

Cases where most rooms have high degree and a small number of rooms have low degree are not a problem, because we are judged on relative error, and the relative error in this case is low. If the average degree is 100 but we estimate that it is 103, we would only be 3% from the answer. But if the average degree is 5 and we estimate that it is 2, we would be 60% away from the answer!

So consider a troublesome case where most rooms have low degree, and a small set of rooms have high degree — few enough that we are unlikely to find one of them by teleporting randomly. If these rooms contribute a significant fraction of the total number of passages, then they must connect to a significant fraction of the total set of rooms. So we can find these with high probability by repeatedly teleporting to a random room and then walking through a random passage with the "W" command.

Consider a series of rounds where we alternate between teleporting to a random room with the "T" command and walking through a random passage with the "W" command. We cannot simply extrapolate the average degree for the whole cave from the average degree of all the rooms we've seen — the rooms we reach with "W" commands are not a uniform sample. We can, however, use the average degree of just the rooms we visited with the "T" command as our estimate for all the rooms we haven't visited, and then add the known degrees to that. This is sufficient to solve the problem.

Another solution is to use an alternating series of "T" and "W" commands as above, and then use a technique called [importance sampling](#). Each visit to a room gives us a sample of the average degree, but each room does not have an equal chance of being visited in any given sample. By weighting each sample appropriately, we can compute a weighted average which serves as an unbiased estimate of the total. The weights are chosen to compensate for the non-uniform probabilities of visiting each room.

When we randomly choose a room and visit it via a "T" command, each room had an equal chance of being chosen; we give this sample a weight of 1. When we visit a room via a "W" command, each room did not have an equal chance of being visited. We need to calculate weights for these samples, such that the expected weight for each room (the probability of visiting that room via a "W" command, multiplied by the expected weight we assign for such visits) is $1/N$. This will give our final estimate the correct expected value.

Consider a sample where we were previously in a room R_1 with degree A , and then we issued a "W" command and walked into a room R_2 with degree B . The probability of us being in R_1 after the last "T" command was $1/N$. The probability of following the passage to R_2 was $1/A$, because R_1 had A passages connected to it. So the overall probability was $1/(AN)$. We assign this sample the weight A/B , so that the contribution to the expected weight for room R_2

is $1/(BN)$. After we sum over all B ways we could arrive at R_2 , we get a total expected weight of $1/N$ for R_2 , as required.

As an example, consider the following interaction:

T	1
1	1
W	
3	2
T	2
2	1
W	
3	2
T	3
3	2
W	
1	1

We get samples of degrees 1, 2, 1, 2, 2, 1 with weights 1, $1/2$, 1, $1/2$, 1, 2, and the weighted average degree is $8/6$.